

Table 2: Signals used by systems administrators

Signal number	Short name	Definition	Purpose
1	HUP	Hangup	Used to report the termination of the controlling process of a terminal. Also used to request process reinitialisation (configuration reload) without termination.
2	INT	Keyboard interrupt	Causes program termination. Can be blocked or handled. Send by typing INTR character (<i>Ctrl-c</i>).
3	QUIT	Keyboard quit	Similar to SIGINT, but also produces a process dump at termination. Send by typing the <i>QUIT</i> character (<i>Ctrl-\</i>).
9	KILL	Kill, un-blockable	Causes abrupt program termination. Cannot be blocked, ignored or handled; always fatal.
15	TERM	Terminate	Causes program termination. Unlike SIGKILL, can be blocked, ignored or handled. The polite way to ask a program to terminate; allows self cleanup.
18	CONT	Continue	Sent to a process to resume, if stopped. Cannot be blocked. Even if handled, always resumes the process.
19	STOP	Stop, un-blockable	Suspends the process. Cannot be blocked or handled.

The *ps* option *j* displays job information, including the initial command shell of each session. Since the sleep example command is currently suspended, the state flag displayed is *T*.

To restart the process in the background, use the *bg* command with the same job ID. The command shell will warn a user who attempts to exit a terminal window (session) with suspended jobs. If the user tries exiting again immediately, the suspended jobs are killed.

Process control using signals

A signal is a software interrupt delivered to a process. Signals report events to an executing program. Events that generate a signal can be an error, or can be external events (like an I/O request or expired timer) or can be generated by explicit requests (e.g., using a signal-sending command or by a keyboard sequence).

Table 2 lists the fundamental signals used by systems administrators for routine process management.

 **Note:** Signal numbers vary on different Linux hardware platforms, but signal names and meanings are standardised. For command use, it is advised to use signal names instead of numbers.

Each signal has a default action, which is usually one of the following:

- *Term* - Causes a program to terminate (exit) at once.
- *Core* - Causes a program to save a memory image (core dump), and then terminate.
- *Stop* - Causes a program to stop executing (suspend) and waits to continue (resume).

A program can be written for expected event signals by implementing handler routines to ignore, replace or extend a signal's default action.

Commands for sending signals by explicit request

Users signal their current foreground process by typing a keyboard control sequence to suspend (*Ctrl-z*), kill (*Ctrl-c*), or core dump (*Ctrl-*) the process. To signal a background process or processes in a different session requires a signal-sending command.

Signals can be specified either by name (*-HUP* or *-SIGHUP*) or by number (like *-1*). Users may kill their own processes but root privileges are required to kill processes owned by others.

- The *kill* command sends a signal to a process using ID. Despite its name, the *kill* command can be used for sending any signal, not just those for terminating programs.
- Use the *killall* command to send a signal to one or more processes with matching selection criteria, such as a command name, processes owned by a specific user or system-wide processes.
- The *pkill* command, like *killall*, can signal multiple processes. *Pkill* uses advanced selection criteria, which can include a combination of the following:
 - *Command* - Processes with a pattern-matched command name
 - *UID* - Processes owned by a Linux user account, effective or real
 - *GID* - Processes owned by a Linux group account, effective or real
 - *Parent* - Child processes of a specific parent process
 - *Terminal* - Processes running on a specific controlling terminal 

 By: Kshitij Upadhyay

The author is RHCSA and RHCE certified, and loves to write about new technologies. He can be reached at upadhyayk04@gmail.com.